



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

Lab Assignment No: 12	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Date of Performance	23 rd July, 2026
Title of Assignment	Kubernetes Pod & Deployment
Theory	Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications. A Pod is the smallest deployable unit in Kubernetes and contains one or more containers that share the same network and storage resources. A Deployment manages Pods by maintaining the desired number of replicas, automatically replacing failed Pods, and supporting updates without downtime. Kubernetes simplifies application management by ensuring high availability, scalability, and efficient resource utilization across a cluster. Commands Used • Checked the Kubernetes client version: kubectl version --client • Verified the Kubernetes cluster information: kubectl cluster-info • Displayed the available cluster nodes: kubectl get nodes • Created the Kubernetes Deployment from the YAML manifest: kubectl apply -f web-deployment.yaml • Listed the running Pods: kubectl get pods Deleted the Kubernetes Deployment and its Pods: kubectl delete -f web-deployment.yaml
Implementation	Step 1: Enable Kubernetes in Docker Desktop

Kubernetes



Enable Kubernetes

Start a Kubernetes single or multi-node cluster when starting Docker Desktop.

Cluster



docker-desktop

kind, 1 node, v1.36.1



Running

Started 9 minutes ago

Reset cluster

Enabled Kubernetes — Enabled Kubernetes in Docker Desktop and verified that the local Kubernetes cluster was successfully initialized.

Step 2: Verify kubectl

```
C:\Users\sanyu>kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

```
C:\Users\sanyu>
```

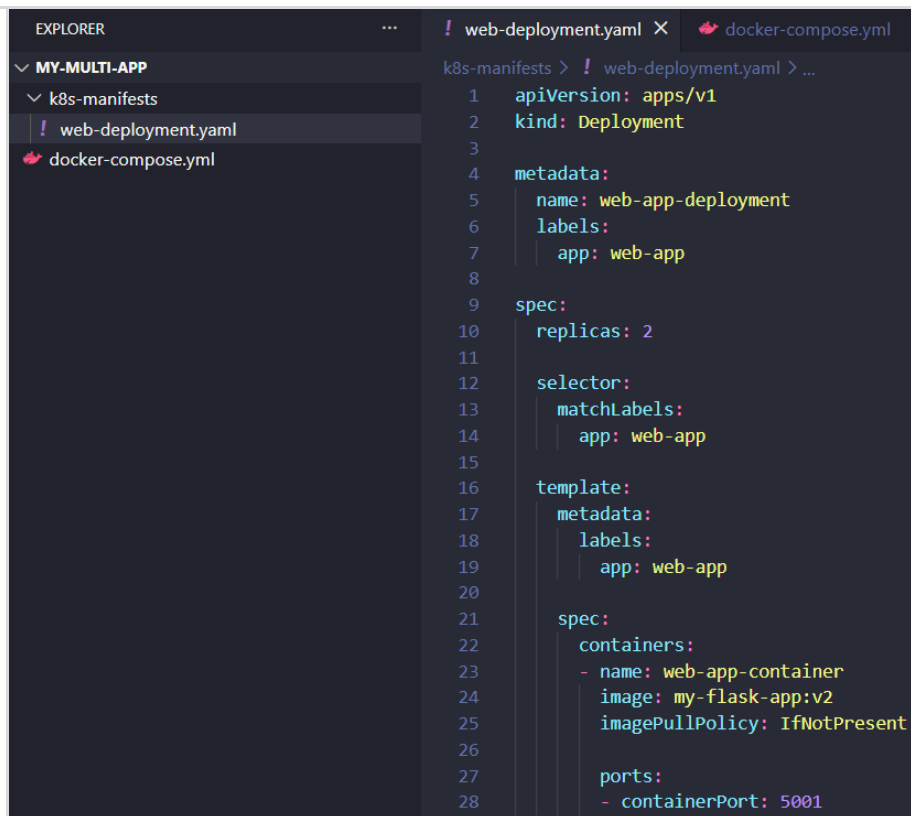
```
C:\Users\sanyu>kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:58703
CoreDNS is running at https://127.0.0.1:58703/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

```
C:\Users\sanyu>kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
```

Verified Kubernetes Cluster — Verified that the local Kubernetes cluster was running successfully using kubectl cluster-info and confirmed that the control-plane node was in the Ready state using kubectl get nodes.

Step 3: Create the Kubernetes Deployment YAML



```
EXPLORER
MY-MULTI-APP
├── k8s-manifests
│   └── web-deployment.yaml
└── docker-compose.yml

k8s-manifests > ! web-deployment.yaml > ...
1  apiVersion: apps/v1
2  kind: Deployment
3
4  metadata:
5    name: web-app-deployment
6    labels:
7      app: web-app
8
9  spec:
10   replicas: 2
11
12   selector:
13     matchLabels:
14       app: web-app
15
16   template:
17     metadata:
18       labels:
19         app: web-app
20
21     spec:
22       containers:
23       - name: web-app-container
24         image: my-flask-app:v2
25         imagePullPolicy: IfNotPresent
26
27       ports:
28       - containerPort: 5001
```

Created Kubernetes Deployment Manifest — Created the web-deployment.yaml file inside the k8s-manifests directory, defining a Kubernetes Deployment with two replicas of the Flask web application using the Docker image my-flask-app:v2.

Step 4: Apply the Deployment Manifest

```
PS C:\Users\riavi\my-multi-app> cd k8s-manifests
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl apply -f web-deployment.yaml
deployment.apps/web-app-deployment created
PS C:\Users\riavi\my-multi-app\k8s-manifests>
```

Applied Deployment Manifest — Applied the Kubernetes Deployment manifest using kubectl apply, creating the Deployment resource in the local Kubernetes cluster.

Step 5: Verify the Pods

```
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web-app-deployment-848b75c97-5qv9x  1/1     Running   0           48s
web-app-deployment-848b75c97-v2668  1/1     Running   0           48s
PS C:\Users\riavi\my-multi-app\k8s-manifests>
```

Verified Running Pods — Verified that the Kubernetes Deployment successfully created two running Pods, each hosting an instance of the Flask web application.

Step 6: Clean Up

```
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl delete -f web-deployment.yaml
deployment.apps "web-app-deployment" deleted from default namespace
PS C:\Users\riavi\my-multi-app\k8s-manifests>
```

Deleted Kubernetes Deployment — Removed the Kubernetes Deployment and the Pods it created using the kubectl delete command, cleaning up the local Kubernetes cluster.

Subject Faculty Name: Prof. Arijit Dutta